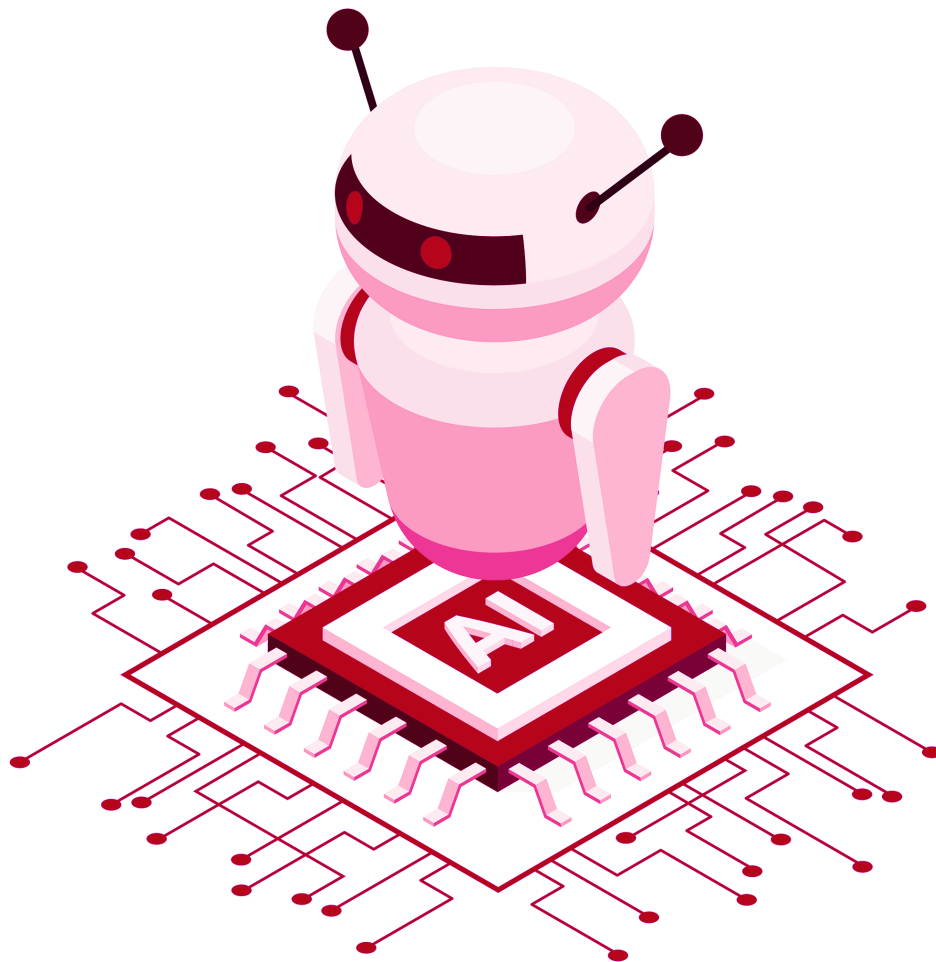




Deep Reinforcement Learning

Professor Mohammad Hossein Rohban

Homework 5:

Model-Based Reinforcement Learning

By:

Parsa Ghezelbash
401110437



Spring 2025

Contents

1 Task 2: Dyna-Q	1
1.1 Task Overview	1
1.1.1 Planning and Learning	1
1.1.2 Experimentation and Exploration	1
1.1.3 Reward Shaping	1
1.1.4 Prioritized Sweeping	1
1.1.5 Extra Points	1
1.2 Questions	2
1.2.1 Experiments	2
1.2.2 Improvement Strategies	2
2 Task 1: Monte Carlo Tree Search	4
2.1 Task Overview	4
2.1.1 Representation, Dynamics, and Prediction Networks	4
2.1.2 Search Algorithms	4
2.1.3 Buffer Replay (Experience Memory)	4
2.1.4 Agent	4
2.1.5 Training Loop	4
2.2 Questions	5
2.2.1 MCTS Fundamentals	5
2.2.2 Tree Policy and Rollouts	5
2.2.3 Integration with Neural Networks	6
2.2.4 Backpropagation and Node Statistics	6
2.2.5 Hyperparameters and Practical Considerations	6
2.2.6 Comparisons to Other Methods	7
3 Task 3: Model Predictive Control (MPC)	8
3.1 Task Overview	8
3.2 Questions	8
3.2.1 Analyze the Results	8

Grading

The grading will be based on the following criteria, with a total of 100 points:

Task	Points
Task 1: MCTS	40
Task 2: Dyna-Q	40 + 4
Task 3: SAC	20
Task 4: World Models (Bonus 1)	30
Clarity and Quality of Code	5
Clarity and Quality of Report	5
Bonus 2: Writing your report in $\text{\LaTeX}$	10

1 Task 2: Dyna-Q

1.1 Task Overview

In this notebook, we focus on **Model-Based Reinforcement Learning (MBRL)** methods, including **Dyna-Q** and **Prioritized Sweeping**. We use the [Frozen Lake](#) environment from [Gymnasium](#). The primary setting for our experiments is the 8×8 map, which is non-slippery as we set `is_slippery=False`. However, you are welcome to experiment with the 4×4 map to better understand the hyperparameters.

Sections to be Implemented and Completed

This notebook contains several placeholders (TODO) for missing implementations as well as some mark-downs (Your Answer:), which are also referenced in section 1.2.

1.1.1 Planning and Learning

In the **Dyna-Q** workshop session, we implemented this algorithm for *stochastic* environments. You can refer to that implementation to get a sense of what you should do. However, to receive full credit, you must implement this algorithm for *deterministic* environments.

1.1.2 Experimentation and Exploration

The **Experiments** section and **Are you having troubles?** section of this notebook are **extremely important**. Your task is to explore and experiment with different hyperparameters. We don't want you to blindly try different values until you find the correct solution. In these sections, you must reason about the outcomes of your experiments and act accordingly. The questions provided in section 1.2 can help you focus on better solutions.

1.1.3 Reward Shaping

It is no secret that [Reward Function Design is Difficult](#) in **Reinforcement Learning**. Here we ask you to improve the reward function by utilizing some basic principles. To design a good reward function, you will first need to analyze the current reward signal. By running some experiments, you might be able to understand the shortcomings of the original reward function.

1.1.4 Prioritized Sweeping

In the **Dyna-Q** algorithm, we perform the planning steps by uniformly selecting state-action pairs. You can probably tell that this approach might be inefficient. [Prioritized Sweeping](#) can increase planning efficiency.

1.1.5 Extra Points

If you found the previous sections too easy, feel free to use the ideas we discussed for the *stochastic* version of the environment by setting `is_slippery=True`. You must implement the **Prioritized Sweeping** algorithm for *stochastic* environments. By combining ideas from previous sections, you should be able to solve this version of the environment as well!

1.2 Questions

You can answer the following questions in the notebook as well, but double-check to make sure you don't miss anything.

1.2.1 Experiments

After implementing the basic **Dyna-Q** algorithm, run some experiments and answer the following questions:

- How does increasing the number of planning steps affect the overall learning process?

Answer: Increasing the number of planning steps allows the agent to simulate more transitions using its internal model without interacting with the environment. This generally accelerates learning by reinforcing known experiences and propagating value updates faster throughout the state space.

- What would happen if we trained on the slippery version of the environment, assuming we **didn't** change the *deterministic* nature of our algorithm?

Answer:

If we use a deterministic Dyna-Q algorithm in a stochastic environment, the learned model will likely be inaccurate, since it only memorizes the first observed transition for each (s, a) pair. As a result, the agents planning will reinforce incorrect assumptions about the environment.

- Does planning even help for this specific environment? How so? (Hint: analyze the reward signal)

Answer:

Yes, planning helps significantly in the deterministic Frozen Lake environment. Since the environment is predictable, once the agent experiences a transition, it can confidently simulate it multiple times through planning. This leads to faster value propagation and reduces the number of real environment steps required to learn an optimal policy. However, if the reward signal is sparse, planning has limited benefit until the agent has experienced a successful episode at least once.

- Assuming it takes N_1 episodes to reach the goal for the first time, and from then it takes N_2 episodes to reach the goal for the second time, explain how the number of planning steps n affects N_1 and N_2 .

Answer:

The number of planning steps n has little to no impact on N_1 , because before the first successful episode, the agent has not yet experienced any meaningful reward. However, n significantly affects N_2 : once the agent reaches the goal once and experiences a positive reward, a higher n allows this reward to propagate more effectively to other relevant states. As a result, N_2 decreases rapidly with higher n values.

1.2.2 Improvement Strategies

Explain how each of these methods might help us with solving this environment:

- Adding a baseline to the Q-values.

Answer:

A baseline can stabilize learning by centering value updates around a reference point. This may reduce variance in updates and encourage better credit assignment, especially in environments with sparse rewards.

- Changing the value of ϵ over time or using a policy other than the ϵ -greedy policy.

Answer:

Reducing ϵ over time allows the agent to explore more in the early stages and exploit its knowledge later. More sophisticated policies can make more informed decisions, balancing exploration and exploitation based on Q-values rather than fixed randomness.

- Changing the number of planning steps n over time.

Answer:

Starting with a small n can prevent overfitting to an initially inaccurate model, while increasing n as the model becomes more accurate allows for better planning and faster convergence. This adaptive approach can yield more stable and efficient learning.

- Modifying the reward function.

Answer:

Reward shaping can guide the agent toward more promising behaviors. For example, assigning small penalties for stepping on frozen tiles or bonuses for reaching intermediate states could accelerate learning and improve exploration. However, shaping must be done carefully to avoid altering the optimal policy.

- Altering the planning function to prioritize some stateaction pairs over others. (Hint: explain how **Prioritized Sweeping** helps)

Answer:

Prioritized Sweeping improves planning efficiency by focusing updates on stateaction pairs that are most likely to impact the value function significantly. This avoids spending planning steps on irrelevant or already learned transitions and allows the agent to converge faster with fewer planning updates.

2 Task 1: Monte Carlo Tree Search

2.1 Task Overview

This notebook implements a **MuZero-inspired reinforcement learning (RL) framework**, integrating **planning, learning, and model-based approaches**. The primary objective is to develop an RL agent that can learn from **environment interactions** and improve decision-making using **Monte Carlo Tree Search (MCTS)**.

The key components of this implementation include:

2.1.1 Representation, Dynamics, and Prediction Networks

- Transform raw observations into **latent hidden states**.
- Simulate **future state transitions** and predict **rewards**.
- Output **policy distributions** (probability of actions) and **value estimates** (expected returns).

2.1.2 Search Algorithms

- **Monte Carlo Tree Search (MCTS)**: A structured search algorithm that simulates future decisions and **backpropagates values** to guide action selection.
- **Naive Depth Search**: A simpler approach that expands all actions up to a fixed depth, evaluating rewards.

2.1.3 Buffer Replay (Experience Memory)

- Stores entire **trajectories** (state-action-reward sequences).
- Samples **mini-batches** of past experiences for training.
- Enables **n-step return calculations** for updating value estimates.

2.1.4 Agent

- Integrates **search algorithms** and **deep networks** to infer actions.
- Uses a **latent state representation** instead of raw observations.
- Selects actions using **MCTS, Naive Search, or Direct Policy Inference**.

2.1.5 Training Loop

1. **Step 1**: Collects trajectories through environment interaction.
2. **Step 2**: Stores experiences in the **replay buffer**.
3. **Step 3**: Samples sub-trajectories for **model updates**.
4. **Step 4**: Unrolls the learned model **over multiple steps**.
5. **Step 5**: Computes **loss functions** (policy, value, and reward prediction errors).

6. **Step 6:** Updates the neural network parameters.

Sections to be Implemented

The notebook contains several placeholders (TODO) for missing implementations.

2.2 Questions

2.2.1 MCTS Fundamentals

- What are the four main phases of MCTS (Selection, Expansion, Simulation, Backpropagation), and what is the conceptual purpose of each phase?

Answer:

1. **Selection:** Starting at the root node, this phase selects a child node recursively using a tree policy (typically UCB) until it reaches a leaf node. The goal is to balance exploration and exploitation while descending the tree.
 2. **Expansion:** If the selected node is not terminal, one or more child nodes are added to the tree. These represent new potential actions or future states not yet explored.
 3. **Simulation (Rollout):** From the newly expanded node, a simulation is conducted to estimate the value of the state.
 4. **Backpropagation:** The value obtained from the simulation is propagated back up the tree to update visit counts and value estimates for each node along the path taken.
- How does MCTS balance exploration and exploitation in its node selection strategy (i.e., how does the UCB formula address this balance)?

Answer:

The Upper Confidence Bound (UCB) formula used in MCTS encourages both high-value nodes (exploitation) and rarely visited nodes (exploration). It typically takes the form:

$$\text{UCB}(s, a) = Q(s, a) + c \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

where $Q(s, a)$ is the average value, $P(s, a)$ is the prior probability, $N(s)$ is the total visits to the parent, and $N(s, a)$ is the number of times action a was taken. The exploration constant c balances the two components.

2.2.2 Tree Policy and Rollouts

- Why do we run multiple simulations from each node rather than a single simulation?

Answer:

Multiple simulations reduce variance in value estimates and allow more accurate statistics to be built up for each node. This improves robustness, helps propagate reliable value information, and ensures that the most promising actions are chosen with confidence.

- What role do random rollouts (or simulated playouts) play in estimating the value of a position?

Answer:

In classical MCTS, random rollouts simulate full episodes from a node to estimate the expected return. This provides a crude but fast value approximation. In Neural MCTS (like MuZero), this is replaced with a learned value function that offers a more informed and consistent estimate.

2.2.3 Integration with Neural Networks

- In the context of Neural MCTS (e.g., AlphaGo-style approaches), how are policy networks and value networks incorporated into the search procedure?

Answer:

The **policy network** provides prior probabilities for actions at each node, guiding the expansion and selection phases. The **value network** predicts the expected return from a state and is used during backpropagation to update node statistics. This integration makes the search process more efficient by using generalization rather than relying on random simulations.

- What is the role of the policy networks output (prior probabilities) in the Expansion phase, and how does it influence which moves get explored?

Answer:

The prior probabilities determine the initial likelihood of selecting each child during selection. These priors bias the search toward actions the policy network believes are replacing uniform random expansion with informed exploration.

2.2.4 Backpropagation and Node Statistics

- During backpropagation, how do we update node visit counts and value estimates?

Answer:

Each node along the simulation path increments its visit count and updates its total value sum. The value estimate Q is typically the average of all returns observed through that node:

$$Q(s, a) = \frac{\text{total value}}{\text{visit count}}$$

- Why is it important to aggregate results carefully (e.g., averaging or summing outcomes) when multiple simulations pass through the same node?

Answer:

Proper aggregation ensures that each node's statistics accurately reflect the outcomes of all simulations that visited it. Without careful averaging, value estimates could become biased or noisy, leading to suboptimal action selection.

2.2.5 Hyperparameters and Practical Considerations

- How does the exploration constant (often denoted c_{puct} or c) in the UCB formula affect the search behavior, and how would you tune it?

Answer:

A higher c increases exploration by emphasizing the prior and penalizing high-visit nodes less. A lower c favors exploitation. This value should be tuned empirically: in sparse-reward environments,

higher exploration may help early learning, while in well-understood environments, lower c accelerates convergence.

- In what ways can the temperature parameter (if used) shape the final move selection, and why might you lower the temperature as training progresses?

Answer:

The temperature controls the sharpness of the final policy distribution derived from visit counts. High temperature encourages exploration (softer distribution), while low temperature encourages exploitation (more deterministic). Lowering temperature over time helps the agent shift from exploration to consistent, optimal play.

2.2.6 Comparisons to Other Methods

- How does MCTS differ from classical minimax search or alpha-beta pruning in handling deep or complex game trees?

Answer:

Minimax and alpha-beta require evaluating every node to fixed depth and depend on a handcrafted evaluation function. MCTS selectively explores the tree, focusing on the most promising branches. This makes MCTS far more efficient in large or complex trees, especially where full depth search is infeasible.

- What unique advantages does MCTS provide when the state space is extremely large or when an accurate heuristic evaluation function is not readily available?

Answer:

MCTS does not rely on a perfect heuristic and builds its value estimates through simulation. This makes it ideal for environments where heuristics are hard to design or inaccurate. Its sampling-based approach allows it to scale to massive state spaces by focusing computation where it matters most.

3 Task 3: Model Predictive Control (MPC)

3.1 Task Overview

In this notebook, we use [MPC PyTorch](#), which is a fast and differentiable model predictive control solver for PyTorch. Our goal is to solve the [Pendulum](#) environment from [Gymnasium](#), where we want to swing a pendulum to an upright position and keep it balanced there.

There are many helper functions and classes that provide the necessary tools for solving this environment using **MPC**. Some of these tools might be a little overwhelming, and that's fine, just try to understand the general ideas. Our primary objective is to learn more about **MPC**, not focusing on the physics of the pendulum environment.

On a final note, you might benefit from exploring the [source code](#) for [MPC PyTorch](#), as this allows you to see how PyTorch is used in other contexts. To learn more about **MPC** and `mpc.pytorch`, you can check out [OptNet](#) and [Differentiable MPC](#).

Sections to be Implemented and Completed

This notebook contains several placeholders (TODO) for missing implementations. In the final section, you can answer the questions asked in a markdown cell, which are the same as the questions in section 3.2.

3.2 Questions

You can answer the following questions in the notebook as well, but double-check to make sure you don't miss anything.

3.2.1 Analyze the Results

Answer the following questions after running your experiments:

- How does the number of LQR iterations affect the MPC?

Answer:

The number of LQR (Linear Quadratic Regulator) iterations controls how thoroughly each MPC optimization step refines the action sequence. More iterations allow for better convergence toward an optimal control trajectory, improving performance and stability of the controller. However, increasing this number also increases computation time. If too few iterations are used, the control sequence may remain suboptimal, resulting in unstable or inefficient behavior.

- What if we didn't have access to the model dynamics? Could we still use MPC?

Answer:

Without access to the true model dynamics, traditional MPC would not be directly applicable because it relies on simulating future states. However, model-free alternatives (like policy gradient methods) or learned dynamics models (using neural networks in model-based RL) can be used instead. In such cases, we can still use MPC by training a predictive model of the environment from data; this is often referred to as *learned MPC* or *model-based planning*.

- Do `TIMESTEPS` or `N_BATCH` matter here? Explain.

Answer:

Yes, both parameters are important:

- `TIMESTEPS` defines how many time steps into the future the MPC controller plans. A longer horizon allows for better long-term planning but increases computational load and may introduce more model error.
- `N_BATCH` determines how many control sequences are optimized in parallel (i.e., batch size). A larger batch allows for better exploration of the control space and faster convergence of the optimal trajectory but also increases memory and compute requirements.

Both must be tuned based on the complexity of the environment and available compute resources.

- Why do you think we chose to set the initial state of the environment to the downward position?

Starting from the downward position (i.e., the pendulum hanging straight down) is the most challenging configuration for the task. The agent must learn a non-trivial control policy to swing the pendulum up and stabilize it, a classic benchmark problem for control. It ensures that the MPC controller is not merely maintaining balance, but actively planning a complex maneuver followed by stabilization.

- As time progresses (later iterations), what happens to the actions and rewards? Why?

As time progresses, the actions tend to become more subtle and smooth, focusing on maintaining the pendulum near the upright position. Initially, large actions are required to swing the pendulum up, resulting in lower rewards due to high energy usage and deviation from the goal. Once upright, the controller refines its actions to minimize energy and stabilize the pendulum, leading to increased and more consistent rewards over time. This transition reflects the success of MPC in both long-term planning and short-term precision control.